

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Навчально-науковий Фізико-Техічний інститут
Кафедра інформаційної безпеки

Лабораторна робота №1
з дисципліни «БЛОКЧЕЙН ТА ДЕЦЕНТРАЛІЗОВАНІ СИСТЕМИ»

Виконав:
ст. групи ФБ-52мп Слепий Роман

Перевірила:
Селюх П.В.

Київ - 2026

Для першого типу лабораторних робіт провести порівняльний аналіз особливостей розгортання систем криптовалют у порівнянні із системою Ethereum. Зробити висновок про можливість чи неможливість взаємозаміни модулів різних систем та пояснити причини.

1. Архітектура ноди Ethereum

Сьогодні одна програма не може працювати як повноцінна нода. Сучасна нода Ethereum обов'язково складається з двох окремих клієнтів, які працюють паралельно та спілкуються між собою через захищений канал за допомогою JWT-токенів. Execution Client відповідає за обробку транзакцій, виконання смарт-контрактів у віртуальній машині (EVM) та управління станом мережі. Найпопулярніші реалізації: Geth, Nethermind, Besu, Erigon. Consensus Client відповідає за механізм Proof of Stake, перевірку блоків та досягнення консенсусу між вузлами мережі. Найпопулярніші реалізації: Prysm, Lighthouse, Teku, Nimbus.

При розгортанні можна обрати тип ноди залежно від завдань. Це буде або Full Node, що зберігає повний стан блокчейну, бере участь у перевірці блоків, але обрізає старі дані стану понад певний ліміт для економії місця. Це стандарт для більшості розробників. Або Archive Node, яка зберігає всі терабайти історичних станів від самого створення мережі – Genesis block. Необхідна для створення блок-експлорерів або складних аналітичних сервісів. Потребує понад 14 ТБ дискового простору. Або ж Light Node, що завантажує лише заголовки блоків і запитує решту даних у повних нод.

На відміну від Bitcoin, Ethereum створює колосальне навантаження на систему введення/виведення (I/O) диска через величезну кількість дрібних операцій читання/запису під час роботи смарт-контрактів. Тільки NVMe SSD об'ємом щонайменше 2 ТБ. Розгортання на HDD або повільних SATA SSD призведе до того, що нода ніколи не зможе наздогнати мережу. Мінімум 16 ГБ ОЗП, але рекомендується 32 ГБ. 4+ фізичних ядра. Стабільний канал від 25 Мбіт/с без лімітів трафіку (нода завантажуватиме та віддаватиме сотні гігабайтів даних на місяць).

ethereum.org/en/developers/docs/nodes-and-clients/run-a-node/
github.com/SomerEsat/ethereum-staking-guides

2. Ethereum vs Bitcoin

Розгортання повної ноди Bitcoin (найчастіше використовується офіційний клієнт Bitcoin Core) є значно простішим і менш ресурсоємним процесом порівняно з розгортанням сучасного Ethereum. Архітектура клієнта є монолітною. Одна програма відповідає і за мережеву взаємодію, і за перевірку правил консенсусу Proof of Work, і за зберігання локальної копії блокчейну. Головна мета такої ноди – забезпечити максимальну фінансову автономію користувача та незалежну валідацію того, що ніхто не порушує базові правила мережі.

Апаратні вимоги до Bitcoin-ноди є досить демократичними, що дозволяє запускати її навіть на міні-комп'ютерах на кшталт Raspberry Pi 4. Для повноцінної роботи без обрізання історії достатньо 2–4 ГБ оперативної пам'яті та близько 600–700 ГБ дискового простору. Хоча використання SSD-накопичувача значно прискорить роботу, Bitcoin Core цілком успішно може функціонувати і на звичайному жорсткому диску, оскільки структура даних UTXO не створює такого екстремального навантаження на систему введення-виведення, як смарт-контракти. Для економії місця можна увімкнути режим "pruned node", якому вистачить лише 10–15 ГБ на диску.

Найважчим етапом розгортання є початкове завантаження блоків (Initial Block Download – IBD). Під час цього процесу нода завантажує та криптографічно перевіряє кожну транзакцію, здійснену з моменту запуску мережі у 2009 році. Залежно від потужності процесора, швидкості диска та інтернет-з'єднання, цей етап може тривати від кількох днів до тижня. Лише після завершення IBD нода вважається повністю синхронізованою і може перевіряти нові транзакції в режимі реального часу.

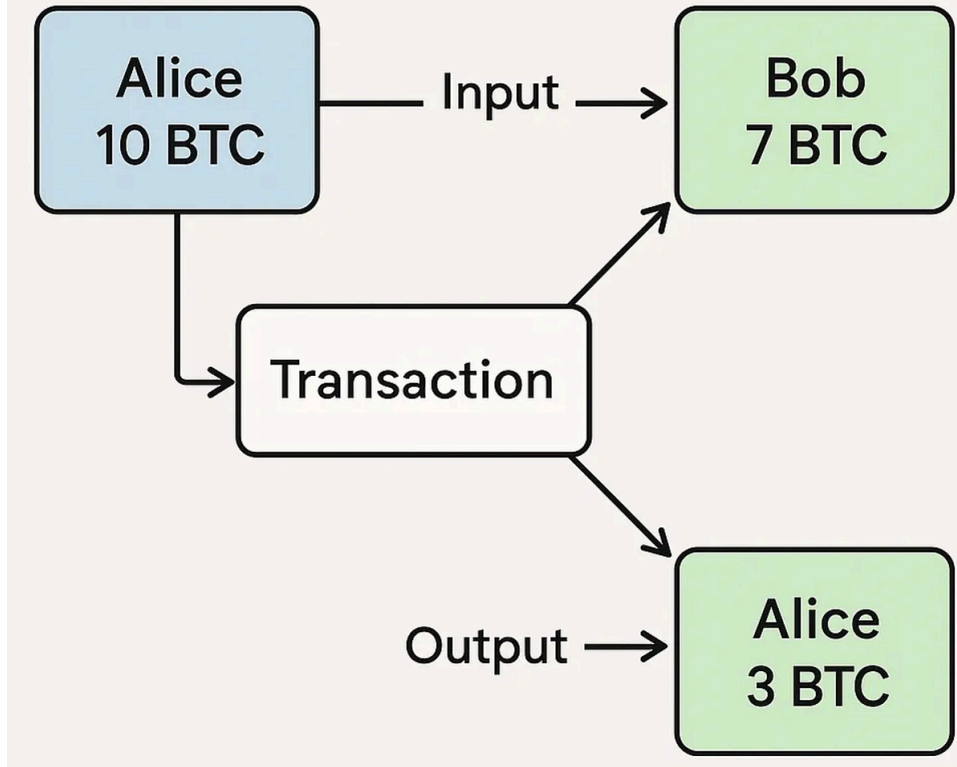
Характеристика	Bitcoin	Ethereum
Архітектура клієнта	Монолітна	Розділена
Алгоритм консенсусу	Proof of Work (PoW)	Proof of Stake (PoS)
Модель обліку даних	UTXO	Account-based
Мінімальні вимоги до RAM	2 ГБ (рекомендовано 4 ГБ)	16 ГБ (рекомендовано 32 ГБ)
Вимоги до накопичувача	~600-700 ГБ (підходить HDD або повільний SSD)	> 2 ТБ (виключно швидкий NVMe SSD)
Швидкість синхронізації	Від кількох днів до тижня	Від кількох годин (завдяки Snap sync) до кількох тижнів (для архівної ноди)

Складність налаштування	Низька	Висока
Головне призначення ноди	Перевірка транзакцій, забезпечення фінансового суверенітету	Виконання смарт-контрактів, стейкінг, підтримка Web3/dApps

Пряма взаємозаміна програмних модулів між вузлами Bitcoin та Ethereum є технічно неможливою через фундаментальні відмінності в їхній архітектурі та базових цілях. Біткоїн створювався як надійна децентралізована платіжна система, тоді як Ефіріум від початку розроблявся як глобальна платформа для виконання смарт-контрактів. Ця різниця зумовила використання різних моделей зберігання даних. Мережа Bitcoin працює на основі моделі невитрачених виходів транзакцій, де баланс динамічно збирається з окремих «шматочків» попередніх переказів. Натомість Ethereum використовує модель облікових записів із фіксованим глобальним станом, що критично необхідно для збереження даних додатків. Через це модуль обробки транзакцій однієї системи фізично не здатен коректно зчитати або модифікувати базу даних іншої.

Крім того, ці блокчейни покладаються на повністю несумісні механізми консенсусу, середовища виконання та мережеві протоколи. Біткоїн використовує алгоритм Proof of Work і максимально спрощену мову Script для базових транзакцій, тоді як сучасний Ethereum працює на базі Proof of Stake і має складну віртуальну машину (EVM), здатну обробляти будь-яку програмну логіку. На мережевому рівні вузли також спілкуються за допомогою різних стандартів, тому пакети даних однієї мережі сприйматимуться іншою просто як незрозумілий інформаційний шум.

Bitcoin UTXO Model



картинка і інформація з

<https://medium.com/@ianolpx/understanding-the-utxo-process-in-bitcoin-fe8db11bedf2>

Алгоритм UTXO (Unspent Transaction Output, або невитрачені виходи транзакцій) – це ключова структура даних, яка відстежує всі доступні для витрачання кошти в блокчейні. Вона є критично важливою для перевірки транзакцій та запобігання подвійному витрачання. На відміну від Ethereum, Bitcoin не використовує традиційну систему балансів рахунків. Замість цього гаманці та оглядачі блоків сканують мережу і підсумовують усі UTXO, прив'язані до конкретної адреси, щоб відобразити користувачеві його зрозумілий баланс.

Ця модель забезпечує кілька вагомих переваг. По-перше, це ефективність, оскільки повним нодам не потрібно зберігати всю історію транзакцій для роботи – достатньо лише актуального набору UTXO. По-друге, це безпека, оскільки кожна транзакція перевіряється незалежно.

По-третє, це конфіденційність, адже часта зміна адрес ускладнює відстеження поведінки користувачів порівняно з моделлю облікових записів. І хоча архітектура UTXO ускладнює створення смарт-контрактів та складної фінансової логіки, глибоке розуміння цієї механіки є обов'язковим для розробки гаманців чи транзакційних систем на базі Bitcoin.

3. Ethereum vs Litecoin

Розгортання повної ноди Litecoin технічно є майже ідентичним до процесу запуску вузла Bitcoin, оскільки ця мережа створювалася як його форк з певними модифікаціями. Для роботи найчастіше використовується монолітний клієнт Litecoin Core, який самостійно виконує мережеву комунікацію, валідацію транзакцій та зберігання бази даних на основі архітектури UTXO. Головна відмінність полягає у використанні алгоритму хешування Scrypt (замість SHA-256) та вчетверо швидшій генерації блоків (кожні 2,5 хвилини), що робить мережу швидшою для платежів. Вимоги до апаратного забезпечення залишаються дуже демократичними: для повноцінної роботи повної ноди достатньо 2–4 ГБ оперативної пам'яті та близько 150 ГБ вільного місця на диску (система цілком нормально працює навіть на звичайних HDD). Процес початкового завантаження блоків (IBD) відбувається значно швидше, ніж у Bitcoin чи Ethereum, і зазвичай займає від кількох годин до доби, не вимагаючи складних конфігурацій чи розгортання додаткових клієнтів.

Характеристика	Litecoin	Ethereum
Архітектура клієнта	Монолітна	Розділена
Алгоритм консенсусу	Proof of Work (PoW на базі Scrypt)	Proof of Stake

Модель обліку даних	UTXO	Account-based
Мінімальні вимоги до RAM	2 ГБ (рекомендовано 4 ГБ)	16 ГБ (рекомендовано 32 ГБ)
Вимоги до накопичувача	~150 ГБ (підходить HDD або повільний SSD)	> 2 ТБ (виключно швидкий NVMe SSD)
Швидкість синхронізації	Від кількох годин до доби	Від кількох годин до тижнів
Складність налаштування	Низька (встановив, запустив, дочекався синхронізації)	Висока (генерація JWT-токенів, налаштування двох сервісів)
Головне призначення ноди	Швидка перевірка транзакцій, підтримка платіжної мережі	Виконання смарт-контрактів, стейкінг, підтримка Web3/dApps

4. Ethereum vs Dash

Розгортання вузла Dash базується на кодовій базі Bitcoin, проте воно суттєво складніше через наявність дворівневої архітектури мережі. Перший рівень забезпечують звичайні майнери, які відповідають за базову

емісію та безпеку ланцюга, тоді як другий рівень складається з Мастернод – спеціалізованих вузлів, що виконують критичні сервісні функції, такі як миттєві та конфіденційні транзакції. Для запуску Мастерноди адміністратор повинен не лише налаштувати сервер із цілодобовим аптаймом та статичною IP-адресою, а й надати заставу в розмірі 1000 DASH, що робить такий вузол акціонером мережі з правом голосу в системі управління бюджетом.

З технічної точки зору, процес ініціалізації вузла передбачає роботу з файлом конфігурації `dash.conf`, де прописуються мережеві параметри та налаштування для взаємодії з мастернод-шаром. Генерація нових блоків у мережі відбувається приблизно кожні 2,5 хвилини, що у чотири рази швидше за Bitcoin, проте вимоги до дискового простору та оперативної пам'яті для звичайного вузла залишаються помірними – близько 4 ГБ ОЗП та 40 ГБ на диску. Така структура дозволяє Dash ефективно поєднувати безпеку традиційного майнінгу з розширеним функціоналом децентралізованої автономної організації (DAO).

Характеристика	Dash	Ethereum
Архітектура клієнта	Дворівнева мережа (Майнери на першому рівні + Мастерноди на другому рівні, клієнт Dash Core)	Розділена
Алгоритм консенсусу	Гібридний: Proof-of-Work (алгоритм X11) + Proof-of-Service	Proof-of-Stake

Модель обліку даних	UTXO	Account-based
Мінімальні вимоги до RAM	4 ГБ (достатньо для повноцінної роботи)	16 ГБ (рекомендується 32 ГБ)
Вимоги до накопичувача	~40 ГБ (підійде звичайний SSD)	> 1-2 ТБ (виключно високошвидкісний NVMe SSD)
Швидкість синхронізації	Швидка (від кількох годин до доби завдяки невеликому розміру блокчейну)	Від кількох годин (за допомогою Snap sync) до кількох тижнів
Складність налаштування	Середня для звичайної ноди; Висока для Мастерноди (вимагає заморозки 1000 DASH, статичного IP та налаштування VPS)	Висока (вимагає генерації JWT-токенів, налаштування та синхронізації двох окремих сервісів)
Головне призначення ноди	Забезпечення миттєвих та анонімних транзакцій (InstantSend/PrivateSend), управління бюджетом DAO	Виконання смарт-контрактів, стейкінг ефіру, підтримка екосистеми Web3 та

		децентралізованих додатків
--	--	-------------------------------

Взаємозаміна модулів між Dash та Ethereum є технічно неможливою через фундаментальний розрив у моделях обробки даних та архітектурі мережі. Dash покладається на структуру UTXO. Спроба інтегрувати модуль обробки транзакцій Dash в Ethereum призвела б до того, що система не змогла б зчитати стан смарт-контрактів, а модуль Ethereum не зрозумів би логіку «розщеплення» транзакції на входи та виходи.

Крім того, модулі консенсусу цих систем мають абсолютно різну логіку прийняття рішень: гібридний механізм з маєрнoдами в Dash архітектурно несумісний із Beacon Chain та валідаторами в Ethereum. Вузол Dash не має доступу до реєстру валідаторів Ethereum і не розуміє механізмів покарання за маніпуляції, а модуль керування ресурсами Dash, який розподіляє комісії між майнерами, маєрнoдами та бюджетом, не зміг би функціонувати в середовищі Ethereum, де газ спалюється або йде валідаторам. Навіть на рівні мережевої комунікації пакети даних будуть відхилятися, оскільки Dash використовує розширений P2P-протокол Bitcoin, а Ethereum – зашифрований стек devp2p на основі таблиць Kademlia.

5. Ethereum vs NEO

Розгортання вузла NEO суттєво відрізняється від екосистеми Ethereum через використання механізму делегованого консенсусу dBFT (Delegated Byzantine Fault Tolerance). У цій системі замість широкої мережі валідаторів існують спеціалізовані консенсусні вузли Bookkeepers, які обираються власниками токенів для формування блоків. Процес консенсусу передбачає призначення одного вузла-спікера, який пропонує блок, після чого дві третини інших вузлів мають підтвердити його для миттєвої та незворотної фіналізації запису. Це забезпечує дуже високу пропускну здатність мережі, яка може сягати 1000 транзакцій на секунду, та виключає можливість розгалуження ланцюга.

Архітектурно NEO використовує модель облікових записів та унікальну систему з двома токенами, де токен NEO надає право на

управління мережею, а токен GAS використовується як паливо для оплати обчислювальних операцій у віртуальній машині. Для розгортання спостерігаючого вузла потрібні помірні ресурси, зокрема близько 4 ГБ оперативної пам'яті та приблизно 20 ГБ дискового простору, оскільки база даних стану мережі оптимізована для швидкості за допомогою RocksDB або LevelDB. Однією з ключових переваг платформи є мультимовна підтримка смарт-контрактів, що дозволяє системним архітекторам інтегрувати рішення, написані на C#, Python або Java, без необхідності переходу на специфічні мови на кшталт Solidity .

Характеристика	NEO	Ethereum
Архітектура клієнта	Монолітна (клієнт neo-cli на базі C#)	Розділена
Алгоритм консенсусу	dBFT (Delegated Byzantine Fault Tolerance)	Proof-of-Stake
Модель обліку даних	Account-based	Account-based
Мінімальні вимоги до RAM	4 ГБ	16 ГБ (рекомендується 32 ГБ)

Вимоги до накопичувача	~20 ГБ (висока ефективність RocksDB)	> 1-2 ТБ (виключно високошвидкісний NVMe SSD)
Швидкість синхронізації	Дуже висока (завдяки dBFT та малій базі)	Від кількох годин (за допомогою Snap sync) до кількох тижнів
Складність налаштування	Середня (ініціалізація та вибір seed-вузлів)	Висока (вимагає генерації JWT-токенів, налаштування та синхронізації двох окремих сервісів)
Головне призначення ноди	Обробка цифрових активів та "розумна економіка"	Виконання смарт-контрактів, стейкінг ефіру, підтримка екосистеми Web3 та децентралізованих додатків

6. А як?

Кросчейн-мости. Блокування оригінального активу у його рідній мережі та випуск його обгорнутої копії в Ethereum-токен wBTC або подібний, яким вже можуть керувати смарт-контракти.

Атомарні свопи. Використання хешованих контрактів для безпечного та прямого обміну активами між абсолютно різними блокчейнами без посередників.

Оракули. Застосування децентралізованих сервісів, які зчитують стан або транзакції в одній мережі і передають ці дані як достовірний факт у смарт-контракт Ethereum.

Сайдчейни та L2-рішення. Створення паралельної мережі-надбудови, яка використовує базову безпеку однієї монети, але запускає всередині себе емулятор віртуальної машини Ethereum. Приклад: мережа Rootstock, яка додає смарт-контракти Ethereum поверх біткоїна.

Висновки:

Розгортання вузлів різних блокчейн-систем суттєво відрізняється за технічними вимогами та архітектурою. Для Ethereum необхідне потужне обладнання через складну структуру стану та обов'язкову роботу двох паралельних клієнтів – виконання та консенсусу. Натомість системи сімейства Bitcoin та Litecoin є менш вимогливими до ресурсів, мають монолітну структуру та дозволяють використання звичайних HDD. Dash та NEO запроваджують додаткові рівні управління у вигляді мастернод або спеціалізованих консенсусних вузлів як Bookkeepers, що потребує специфічних налаштувань застави та вибору довірених вузлів.

Головний архітектурний розрив полягає у несумісності моделей даних, механізмів консенсусу та економічних моделей. Системи на основі UTXO (Bitcoin, Dash, Litecoin) оперують залишками транзакцій, тоді як Account-based платформи (Ethereum, NEO) працюють зі станом облікових записів, що є критичним для функціонування смарт-контрактів. Використання різних віртуальних машин (як EVM у Ethereum) та обмежених мов скриптів робить пряму заміну програмних модулів неможливою, оскільки кожен компонент жорстко прив'язаний до специфічних правил безпеки та методів розрахунку комісій своєї екосистеми.

Подолання цієї ізоляції реалізується не через зміну внутрішніх модулів, а за допомогою зовнішніх протоколів взаємодії. Кросчейн-мости дозволяють переносити активи між мережами через механізм блокування та випуску обгорнутих токенів, а атомарні свопи забезпечують прямий

безпечний обмін між блокчейнами. Додатково використовуються децентралізовані оракули для передачі даних між системами та сайдчейни, які створюють сумісні з Ethereum середовища виконання, що дозволяє об'єднувати переваги різних мереж на зовнішньому рівні.